

Leveraging Phase Polynomials for Quantum Circuit Optimization

Zihan Chen
Rutgers University
Piscataway, NJ, USA
zihan.chen.cs@rutgers.edu

Henry Chen
Rutgers University
Piscataway, NJ, USA
hc867@rutgers.edu

Yuwei Jin
Rutgers University
Piscataway, NJ, USA
jyw413482880@gmail.com

Enhyeok Jang
Yonsei University
Seoul, Korea
enhyeok.jang@yonsei.ac.kr

Mingkuan Xu
Carnegie Mellon University
Pittsburgh, PA, USA
mingkuan@cmu.edu

Vannessa Chan
Rutgers University
Piscataway, NJ, USA
vlc74@rutgers.edu

Won Woo Ro
Yonsei University
Seoul, Korea
wro@yonsei.ac.kr

Eddy Z. Zhang
Rutgers University
Piscataway, NJ, USA
eddy.zhengzhang@gmail.com

Abstract—Quantum circuits on resource-limited hardware require optimizing regions dominated by $\{\text{CNOT}, R_z\}$, which account for a large fraction of operations and often dominate execution cost. This optimization can be challenging because phase-polynomial blocks are fragmented by basis-changing gates such as H , and optimizing phase parities alone may increase the cost of downstream basis transformations. Existing phase-polynomial approaches are limited to single-block or phase-only optimization, while subcircuit rewriting approaches are local and scale poorly beyond small rewrite windows. We introduce *PhasePoly*, a compiler optimization pass that jointly optimizes phase-parity and output-parity networks and employs a cross-block intermediate representation to reuse parities across phase-polynomial block barriers. This approach is effective because its unified parity-matrix representation exposes long-range $\{\text{CNOT}, R_z\}$ structure that local rewriting and single-block methods cannot capture. *PhasePoly* reduces total gate count by up to 50.00% (34.70% on average) and CNOT count by up to 48.57% (26.83% on average), while scaling to large circuits and improving both fault-tolerant compilation and near-term hardware execution. *PhasePoly* is available at <https://github.com/ruadapt/PhasePoly>.

Index Terms—Phase Polynomial Optimization, Quantum Circuit Optimization, Cross-block Intermediate Representation.

I. INTRODUCTION

Quantum computing has gained increasing attention for its potential to address problems that are intractable for classical computers, including integer factorization [1], discrete logarithms [2], database search [3], and simulations in physics and chemistry [4], [5]. However, current quantum hardware remains constrained by limited time-space volume, making efficient algorithm design and program optimization essential.

Circuit optimization is a technique that transforms an input circuit into a semantically equivalent but more efficient form. Its primary goal is to reduce gate count and/or circuit depth, thereby lowering error rates, shortening execution time, and improving fidelity. Circuit optimization is an important part of industrial-strength software frameworks, such as in Qiskit [6], Quilc [7], and TKET [8].

Our work targets *phase polynomial optimization*, a class of transformations over circuits composed of CNOT and R_z gates. In such circuits, CNOT gates compute XOR parities

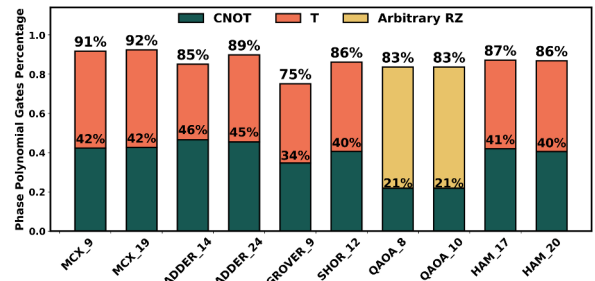


Fig. 1: Breakdown of $\{\text{CNOT}, R_z\}$ usage in representative quantum circuits, showing that a large fraction of gates fall into $\{\text{CNOT}, R_z\}$ regions corresponding to phase polynomial subcircuits. The R_z terms are further decomposed into $\{Z, S, T\}$, as R_z subsumes both Clifford and non-Clifford rotations. The two QAOA benchmarks represent noisy-circuit instances and therefore have arbitrary R_z decompositions.

of input qubits, while R_z gates apply phase rotations conditioned on these parities. The notion of *phase polynomials* was introduced by Amy *et al.* [9] in the *sum-over-paths* form (see Section II), an efficient intermediate representation (IR) that expresses CNOT + R_z circuits as parity-controlled R_z rotations followed by output basis transformations. This representation has enabled logical circuit optimization [10], [11], equivalence checking [12], [13], and hardware-aware circuit synthesis [14]–[16].

Why Are Phase Polynomials Important? Phase polynomial subcircuits occur extensively in quantum circuits. To quantify this, we evaluate a representative set of benchmarks commonly used in circuit optimization studies [10], [11], [17]–[19], including multi-controlled NOT (MCX) circuits [20], [21], Grover’s search [3], Shor’s factoring algorithm [22], the quantum approximate optimization algorithm (QAOA [23]), and Hamiltonian dynamics (HAM). As shown in Fig. 1, more than 75% of the gates in these benchmarks belong to $\{\text{CNOT}, R_z\}$ regions, with several cases exceeding 90%.

Relationship to Fault-Tolerant Quantum Computing: In fault-tolerant (FT) quantum computing, a standard universal gate set is Clifford+ T , where Clifford operations are typically generated by $\{\text{CNOT}, H, S\}$. Within this setting, phase polynomial regions naturally arise from the combination of CNOT gates and diagonal phase rotations (e.g., Z , S , and T), which together capture both parity computation and phase accumulation. As shown in Fig. 1, these regions dominate a wide range of FT-oriented benchmarks, where CNOT and T gates account for the majority of operations and appear in comparable proportions.

Previously, FT optimization has focused primarily on reducing T gates due to their high cost under magic-state distillation [24]. However, recent advances in magic-state cultivation [25]–[27] and updated resource models [28] indicate that the costs of T gates and CNOTs are becoming increasingly comparable. This shift highlights the need for optimization techniques that jointly reduce both gate types. In particular, it motivates treating phase polynomial regions as first-class optimization targets, as they directly capture the dominant cost structure in both FT and near-term noisy circuits.

How Are Phase Polynomials Integrated into General Circuit Optimization Frameworks? Current approaches use phase polynomials as auxiliary tools for local circuit-rewriting optimizers. For instance, Quartz [17] uses rotation merging (a subset of phase polynomial techniques) as a preprocessing step. Quartz automatically searches and constructs equivalent circuit classes (ECCs) for local subcircuit rewriting—replacing a sub-circuit with a better one in its ECC class. QUESO [18] is also a rewriting framework, which uses phase polynomials to enhance ECC generation via a polynomial identity filter (PIF)—creating more circuits in ECCs for rewriting purposes.

Phase polynomials are used as auxiliary tools, rather than a standalone optimization pass. This is due to the lack of a unified intermediate representation that addresses not only a single block of CNOT + R_z gates, but also transitions between blocks and to other non-phase-polynomial gates. Amy *et al.* [10] propose the Gray-Synth algorithm, which greedily orders and synthesizes phase terms according to the Gray code. However, its theory applies only to single blocks and only to their phase-rotation components. It does not systematically handle how XOR propagation of phase terms interacts with the block’s output-basis transformation—a crucial requirement for stitching phase polynomials correctly into a full circuit. Because Gray-Synth is restricted to individual phase polynomial blocks, it cannot operate on general quantum circuits with non-phase-polynomial gates, limiting its applicability as a general-purpose circuit optimization pass.

How Our Work Differs from Prior Work: We present *PhasePoly*, a compiler optimization pass that elevates phase polynomial optimization from an auxiliary technique to a first-class stage in general circuit compilation. The key insight is that phase polynomial structure enables global reasoning beyond local rewriting, if both phase and parity transformations are modeled in a unified and extensible way.

1. Beyond the phase-parity: A Unified Representation

for Comprehensive Phase Block Analysis. Within a single $\{\text{CNOT}, R_z\}$ block, prior methods primarily minimize the cost of the phase-parity network while treating the output basis transformation as a separate synthesis problem. This separation misses co-optimization opportunities: different realizations of the same phase terms can induce different downstream costs. We address this by introducing a unified representation that captures both phase and output parities under the same CNOT transformations, enabling coordinated optimization.

2. Breaking the Single-Block Barrier: Cross-Block Optimization. Phase polynomial regions in general circuits are partitioned by basis-changing gates (e.g., H), limiting existing approaches to short-range, block-local improvements. We overcome this limitation by introducing a cross-block intermediate representation and optimization that enables long-range transformations beyond CNOT, R_z subcircuits. As illustrated in Fig. 4, H gates partition the circuit into three phase polynomial blocks. Prior approaches, such as Gray-Synth, are limited to block-local optimization, whereas our cross-block approach jointly optimizes non-adjacent blocks (e.g., the first and third), uncovering optimization opportunities beyond block boundaries while preserving correctness and efficiency.

3. Standing Alone, Working Together: Orthogonal Integration with Other Frameworks. Across diverse benchmarks—including arithmetic circuits, multi-controlled Toffoli gates, Hamming coding functions [29], Hamiltonian simulation, QAOA, Grover’s algorithm, and Shor’s algorithm—we show strong standalone performance of *PhasePoly*. We also demonstrate complementary benefits when combining *PhasePoly* with existing rewriting frameworks. Our results demonstrate that phase polynomial optimization should be treated as a first-class stage in the compilation pipeline: not merely an auxiliary to subcircuit rewriting frameworks, but a collaborator that exposes otherwise unreachable opportunities.

Our contributions are summarized as follows:

- *Systematic revisiting of phase polynomials:* We provide the first systematic investigation of phase polynomials in general circuit optimization, establishing their necessity as a standalone optimization pass.
- *Holistic phase polynomial optimization:* We introduce a framework that jointly optimizes the phase rotation and output basis transformation. Combined with cross-block optimization, it overcomes single-block limitations and enables substantially stronger results.
- *Extensibility and scalability:* Unlike fixed-size subcircuit rewriting methods, our approach scales naturally to large circuits and demonstrates strong extensibility across diverse benchmarks. Our approach delivers significant reductions in total gate count (up to 50.00%, average 34.70%), CNOT gates (up to 48.57%, average 26.83%).
- *Orthogonality:* *PhasePoly* is orthogonal to subcircuit rewriting. While rewriting may perform comparably or better on small circuits, our approach scales more effectively and uncovers additional improvement (up to 13% for already highly optimized circuits). Together, they close both short- and long-range optimization gaps.

II. BACKGROUND AND KEY INSIGHTS

A quantum circuit is a sequence of quantum gates acting on an n -qubit system. The computational basis states are written as $|x\rangle$ with $x \in \mathbb{F}_2^n$, which is a binary vector. A CNOT acting on control x and target y maps $|x, y\rangle$ to $|x, x \oplus y\rangle$. An example in Fig. 2 illustrates how CNOTs update parities and how R_z gates must track the XOR sums to preserve the correct output basis. The transformed circuit is functionally equivalent to the original while eliminating 1 CNOT and 2 T gates.

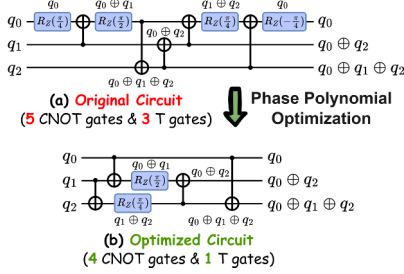


Fig. 2: Phase polynomial optimization example: (a) and (b) are functionally equivalent; however, (a) uses 5 CNOTs and 3 T gates, whereas (b) uses 4 CNOTs and 1 T gate.

A *phase-polynomial circuit* is a circuit region composed solely of $\{\text{CNOT}, R_z\}$ gates. Such regions are not universal for general quantum circuits. In a general circuit, the appearance of non-phase-polynomial gates (e.g., H gates) changes the computational basis and therefore terminates the region. We define a *phase-polynomial block* as a maximal contiguous sub-circuit containing only $\{\text{CNOT}, R_z\}$ gates in general circuits. Formally, one can represent a phase polynomial circuit in a *sum-over-paths* form [9]–[11] such that

$$U |x_1, \dots, x_n\rangle = e^{ip(x_1, \dots, x_n)} |g(x_1, \dots, x_n)\rangle \quad (1)$$

where $p(x)$ is a Boolean polynomial over XOR parities with phase coefficients, and $g(x)$ is an affine reversible transformation implemented by a CNOT network.

$$p(x_1, \dots, x_n) = \sum_{y \in \{0,1\}^n} \theta_i \left(x_1 y_1 \oplus \dots \oplus x_n y_n \right) \quad (2)$$

In Fig. 2(a), the phase function can be written as a weighted sum of parity terms: $p(q_0, q_1, q_2) = \frac{\pi}{4} q_0 + \frac{\pi}{2} (q_0 \oplus q_1) + \frac{\pi}{4} (q_1 \oplus q_2) + \frac{\pi}{4} q_0 = \frac{\pi}{2} (q_0 \oplus q_1) + \frac{\pi}{4} (q_1 \oplus q_2)$. Each term corresponds to a phase rotation conditioned on a parity of input variables. In general, a **phase-parity** is the XOR of a subset of input qubits, and a **phase-parity function** $p(x)$ is a linear combination of such parities with rotation angles. At the circuit level, these parities are constructed using CNOT gates and realized by applying $R_z(\theta)$ rotations on the corresponding qubit lines; we refer to this structure as the **phase-parity network**.

The function $g(x)$ represents the **output basis transformation**, a linear reversible mapping of computational basis states implemented by a CNOT network. For example, in Fig. 2(a), $g(q_0, q_1, q_2) = (q_0, q_0 \oplus q_2, q_0 \oplus q_1 \oplus q_2)$. Each output is a parity of the input qubits. We call these parities **output parities**, and the corresponding CNOT circuit implementing this linear transformation the **output-parity network**.

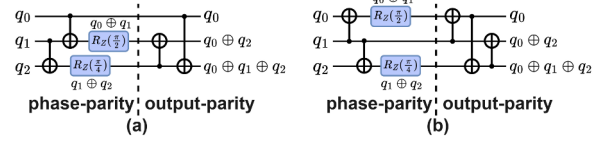


Fig. 3: Two circuits that both implement the p function using the same minimal gate count, but result in different costs for the g function. (a) uses one fewer CNOT than (b).

A. Single-block Optimization: One Stone Two Birds

The phase function p —capturing phase parities—has been extensively studied in the context of phase polynomial optimization [9], [10], [15]. In contrast, the output transformation g —capturing output parities—is typically studied in a different context, namely linear reversible circuit synthesis [30], [31]. No prior work has come up with a way to unify these two problems into one model; they have addressed these two problems separately and solved them one after another.

However, such separate handling may miss co-optimization opportunities. This is because the CNOT network synthesis for the phase parity function affects the parity state of each qubit, which is subsequently used as input to the output parity component—the g function. Thus, implementations that achieve the minimal gate count for the phase parity function may not minimize the gate count for the output parity function.

We show such an example in Fig. 3 where two circuits implement the phase-parity (p) function with the same minimal cost—both using only 2 CNOTs in (a) and (b). However, they lead to different CNOT costs in the basis transformation (g) function—one using two and the other using three. Thus, even if the phase-parity network is individually minimal, ignoring its interaction with the output-parity network leads to non-minimal overall CNOT overhead. **In this paper, we unify these two problems into one, in order to capture their correlation.** For the example in Fig. 3, our framework is able to find the overall minimal transformation cost in (a). The details of the co-optimization are in Section III-A.

B. Breaking the Block Barrier: Long-Range Optimizations

A general circuit may contain non-phase polynomial gates (e.g., H gates) acting as *block barriers* that split a circuit into multiple phase polynomial blocks. Optimizing individual blocks may be insufficient, as each phase polynomial block may be too small. **This raises a key question: can optimization opportunities be exposed across block boundaries?**

We show that the answer is YES. Consider the multi-controlled-NOT gate (MCX) [20], [21], a core primitive in many algorithms and simulations [3], [22], [32], [33], which appears prominently in modular exponentiation—the dominant cost in Shor’s algorithm. We show an MCX implementation with n qubits using the standard 3-qubit Toffoli gates in Fig. 4 (left). This implementation is further decomposed into Clifford and T gates. The H gates act as barriers, forcing phase polynomial optimization to act only within a Toffoli gate block in the traditional phase polynomial context.

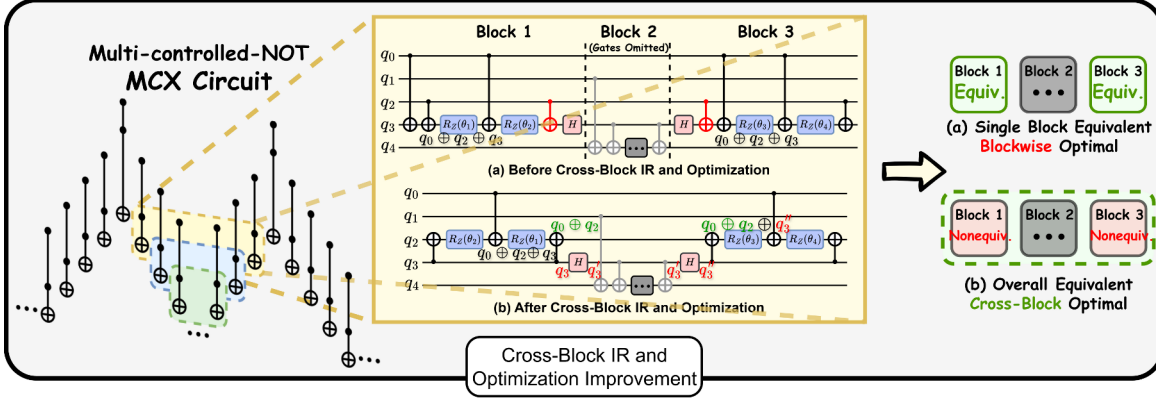


Fig. 4: Phase polynomial cross-block intermediate representation (IR) and optimization on a multi-controlled-NOT (MCX) circuit. *Left*: a portion of an MCX construction, where Toffoli gates are chained in a prescribed order over control, target, and ancilla qubits; the highlighted region (with intermediate gates omitted for clarity) is expanded on the right. *Circuit (a), before*: two H gates act as block barriers, partitioning the circuit into three phase polynomial blocks. Optimizing each block in isolation attains block-local optima (e.g., eight CNOTs and four R_z gates in the shown region) but misses cross-block reductions. *Circuit (b), after*: phase polynomial cross-block IR merges the three blocks into a single phase polynomial region. Cross-block optimization reorders the parity network structure and eliminates two redundant CNOT gates (marked in red in circuit (a)), preserving functional equivalence while lowering the overall CNOT cost.

Traditionally, both local rewriting and single-block phase-polynomial methods reduce a Toffoli block’s CNOT cost from 6 to about 4 on average. Our cross-block approach goes further: by breaking the block barrier, we reduce roughly half of the Toffoli structures to 3 CNOTs on average, achieving a non-constant improvement in CNOT count. In Fig. 4, we show the circuit in the highlighted regions, for which a parity term $q_0 \oplus q_2$ can be reused (generated in block 1 and reused in block 2). This eliminates two CNOT gates across two blocks, reducing the cost by one CNOT per block on average. This opportunity, however, cannot be captured by the traditional approach, as (1) between blocks 1 and 3, there are many other gates—exceeding the local sub-circuit rewriting range, and (2) H gates delineate the single block boundaries.

We further provide quantitative evidence in Fig. 5, showing MCX circuits of increasing size. Our approach achieves greater reductions in both CNOT and total gate counts compared to Quartz [17] and QUESO [18], with the gap widening as the number of qubits increases. This trend arises because the number of Toffoli gates scales with circuit size, increasing the opportunities for cross-block parity reuse. These results show that our approach captures optimization opportunities beyond prior techniques. Details of the cross-block optimization are described in Section III-C.

C. Overcoming the Scaling Challenges

State-of-the-art quantum circuit optimizers largely rely on *equivalent subcircuit rewriting*, where small subcircuits are replaced using precomputed equivalence classes (ECCs). While effective for local transformations, these methods are fundamentally limited by the size of the rewrite rules: practical ECCs typically cover only small patterns (e.g., up to 3 qubits and depth 3–6), as constructing larger equivalence classes becomes computationally intractable. As a result, capturing

long-range optimizations requires many local rewrites and becomes increasingly difficult as circuit size grows.

In contrast, our approach avoids this limitation by operating on a structured parity-matrix representation rather than fixed-size rewrite patterns. By manipulating this representation for both single- and cross-block optimization (see Section III), it scales naturally with circuit size and captures long-range transformations beyond local rewriting.

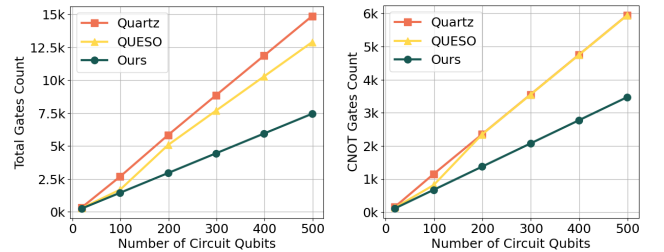


Fig. 5: Gate count comparison of different optimization frameworks on multi-controlled NOT (MCX) circuits.

We compare against subcircuit rewriting frameworks Quartz and QUESO on MCX circuits (Fig. 4, left). Rewriting methods are given a fixed 7200s budget per circuit, while our phase polynomial optimization uses at most 3600s for the largest instances. Within these settings, the performance of local subcircuit rewriting degrades steadily as the number of qubits grows from 19 to 100, with even sharper declines at larger scales (Fig. 5). Since QUESO extends subcircuit rewriting by incorporating phase contributions, its performance is slightly weaker than phase polynomial optimization but consistently stronger than Quartz for circuits with fewer than 100 qubits. However, as circuit scale increases, the gap widens: QUESO’s reduction rate deteriorates, while phase polynomial optimization continues to sustain a linear growth trend.

III. PHASE POLYNOMIAL CIRCUIT OPTIMIZATION

A. Co-Optimization of Phase- and Output-parity Networks

The phase polynomial optimization consists of two components: the *phase-parity* network and the *output-parity* network. Prior work typically optimizes them separately. The output-parity network corresponds to the linear transformation $g(x)$ in Eq. 1. This transformation can be represented as a binary matrix over $\text{GF}(2)$, where each row encodes the parity appearing in the corresponding output qubit.

Each CNOT corresponds to an elementary row operation over $\text{GF}(2)$: a $\text{CNOT}(i, j)$ performs $\text{row}_j \leftarrow \text{row}_j \oplus \text{row}_i$. Thus, a CNOT circuit can be viewed as a sequence of matrix updates that, starting from the identity, produce the output-parity matrix describing $g(x)$.

For example, in Fig. 6, $G_1 \mapsto \text{CNOT}(q_0, q_1)$ updates the row of q_1 to $q_0 \oplus q_1$, represented as $[1, 1, 0, 0]$ in the second row of the matrix G_1 . Conversely, synthesizing a CNOT network for a given $g(x)$ amounts to reducing the matrix back to the identity via elementary matrix multiplications, which can be performed using Gaussian elimination or related linear-algebra techniques [30], [31], which guarantees that the transformation can be realized in polynomial time.

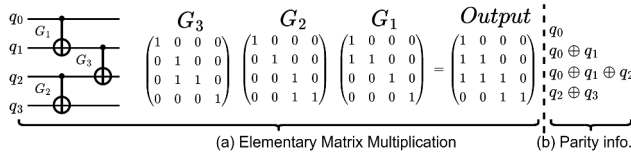


Fig. 6: CNOT gates correspond to elementary row operations over $\text{GF}(2)$. Synthesizing the CNOT network is equivalent to reducing this matrix to the identity via Gaussian elimination.

However, this representation cannot directly capture phase-parity networks with parity-controlled R_z rotations. Unlike the output-parity network, the phase-parity block is not a square transformation matrix: each column represents a parity term rather than an output mapping, and therefore it cannot be reduced to the identity using Gaussian elimination.

To jointly optimize the phase-parity and output-parity networks, we represent both in a unified column-based parity form. Each phase term corresponds to a parity vector (e.g., $(110)^T$ denotes $q_0 \oplus q_1$). The output transformation $g(x)$, typically represented as a square matrix (e.g., Fig. 6), can be transposed so that each output parity is also a column vector. Thus, both $p(x)$ and $g(x)$ are expressed as collections of parity vectors over the input qubits.

We therefore introduce a *coupled parity matrix* representation: $[\text{phase-parity block} \mid \text{output-parity block}]$, to jointly represent phase terms and output parities. Under our convention, $\text{CNOT}(i, j)$ applies $\text{row}_i \leftarrow \text{row}_i \oplus \text{row}_j$ to both blocks simultaneously, coupling their optimizations. Though a physical CNOT updates the target qubit, our operation updates the phase-parity term instead of the quantum state itself, following prior works [10], [15].

Fig. 7 illustrates this representation for the circuit in Fig. 2. When a phase-parity column reaches Hamming weight 1,

its parity depends on a single qubit, so the corresponding R_z rotation can be emitted and the column removed. For example, after $\text{CNOT}(q_1, q_0)$, the column $(110)^T$ becomes $(100)^T$, meaning that the parity $q_0 \oplus q_1$ has been propagated onto qubit 0. Repeating such row operations eliminates phase columns while updating the output-parity matrix. Once all phase columns are removed, the remaining output-parity matrix can be synthesized via Gaussian elimination.

$$\left[\begin{array}{c|ccc} 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{array} \right] \xrightarrow{\text{CNOT}(q_1, q_0)} \left[\begin{array}{c|ccc} 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{array} \right] \xrightarrow{\text{Insert } R_z} \left[\begin{array}{c|ccc} 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{array} \right]$$

Fig. 7: Coupled co-optimization representation. A CNOT simultaneously updates the phase-parity and output-parity block.

B. Overall Phase Polynomial Co-Optimization Framework

This optimization task can be formulated as a **CNOT-minimization parity network synthesis problem** [10], where the optimization objective is to minimize the total gate count. CNOT network synthesis is NP-hard [10], [34] for both phase-parity and output-parity optimization, making it infeasible to guarantee optimal solutions in polynomial time. We therefore adopt a heuristic-based framework to optimize the sequence of CNOT and R_z operations.

We model the search space as a tree in which each node represents a circuit state and each edge corresponds to a CNOT operation (Fig. 8). A root-to-leaf path specifies a sequence of CNOTs. A valid leaf node is reached when the phase-parity matrix becomes empty, and the output-parity matrix reduces to the identity. The optimization objective is to minimize the number of CNOT gates, which corresponds to finding a minimum-cost path in this search tree.

1) *Active Row Pair CNOT Selection*: As described in Section III-A, a CNOT corresponds to an XOR between two rows of the parity matrix, referred to as a **row pair**. The operation is directional: a pair $(\text{row}_i, \text{row}_j)$ represents a CNOT with control qubit i and target qubit j , updating the control row as $\text{row}_i \oplus \text{row}_j$ while leaving the target row unchanged.

A row pair is considered valid if it reduces the Hamming weight of at least one column. Such pairs are called **active row pairs**. However, repeatedly applying the same row pair can cause livelock, preventing progress. To avoid this, we restrict the search using an **active column set**, defined as the subset of columns whose Hamming weights can be reduced. Row pairs are only considered if they help at least one column within the active column set (reducing at least one 1). After a CNOT operation, the active column set may be updated. At initialization, or after eliminating a column, the active column set includes all remaining columns.

2) *The Priority Queue Implementation*: To evaluate the state of each move after applying a CNOT from the active row pair set, we derive the following insights:

- **Phase-parity cost** $h_1(n)$: the total Hamming weight (number of 1s) in the phase-parity matrix correlates with the number of CNOT gates required for phase terms.

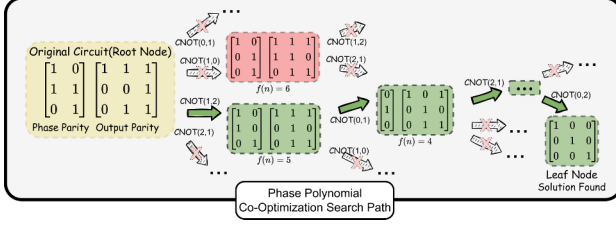


Fig. 8: Search tree for Phase Polynomial Co-Optimization. The root denotes the original circuit, the leaf represents the optimized circuit, and edges represent CNOT operations. Node costs reflect heuristic evaluations of intermediate states.

- **Output-parity cost** $h_2(n)$: the estimated number of CNOTs required to transform the output-parity matrix into the identity using Gaussian elimination.
- **Accumulated cost** $g(n)$: the number of CNOTs already applied along the path from the root to the current node.

The overall cost function is:

$$f(n) = g(n) + h_1(n) + h_2(n) \quad (3)$$

We adopt an A* search with a priority queue ordered by $f(n)$. When a state reaches an empty phase-parity matrix, the synthesis of the output-parity network is completed using Gaussian elimination. Otherwise, new successor states are generated by applying CNOTs from the active row pair set in an effort to empty the phase-parity matrix. An example is illustrated in Fig. 8.

When multiple states share the same cost, we apply a tie-breaking rule based on the lexicographic ordering of

$$[f(n), h_1(n), h_2(n), -g(n)].$$

This rule prioritizes states with lower estimated costs and, secondarily, those closer to completion.

Because the search space grows exponentially, unrestricted A* search becomes impractical. We therefore use a **space-bounded A* search** [35], [36], capping the priority queue size. When the limit is reached, lower-priority nodes are discarded to prune unpromising paths and control memory growth. We further employ a **multiple-solution search** controlled by a hyperparameter k . Each time a goal state is found, it is added to a solution set of size k . The search terminates when either the priority queue is empty or k solutions are obtained, after which the best one is returned.

C. Cross-block Intermediate Representation and Optimization

1) **Static Single-assignment (SSA) Style Rotation Merging**: For the standard Clifford+ T gate set $\{T, T^\dagger, S, S^\dagger, H, X, \text{CNOT}\}$ [37], the phase polynomial gate set $\{\text{CNOT}, R_z\}$ is *not* universal. We therefore partition a general quantum circuit into blocks of phase polynomial subcircuits separated by non- R_z single-qubit gates. In particular, H gates act as *block barriers*: two-qubit gates whose semantics depend on the post- H basis are excluded from the preceding phase polynomial block. Fig. 9(a) illustrates a case with two such blocks.

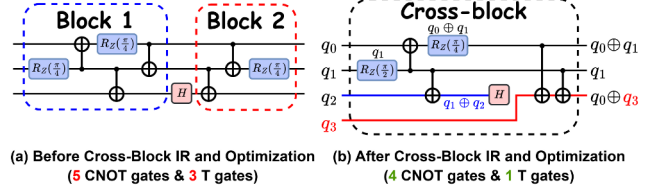


Fig. 9: Cross-block optimization: (a) Two separate blocks before optimization (5 CNOTs, 3 T gates). (b) Across the H gate, we create a new qubit wire. In the new circuit, the output-parity before the H gate is set to $q_1 \oplus q_2$, the same as that before the H gate in the original circuit. Now we have a new phase-polynomial block, by optimizing this new block, we reduce the circuit to 4 CNOTs and 1 T .

To enable rotation merging across block boundaries, we introduce an SSA-style [38] *qubit-state renaming and rotation merging*. Each input qubit state, and each state created after an H gate, is assigned a fresh SSA identifier. Every R_z gate is then tagged with the SSA ID of the qubit state it acts on. By merging all phase terms associated with the same SSA ID, we achieve *whole-circuit* rotation merging rather than being restricted to a single block.

For example, in Fig. 9(b), two T gates on q_1 share the same SSA ID and therefore merge, even though they appear in different blocks. Likewise, the blue wire q_2 ends at the SSA state $\{q_1 \oplus q_2\}$ before the H gate; after the H gate we create a new SSA state q_3 , yielding an updated output-parity $\{q_0 \oplus q_3\}$.

Prior work on rotation merging [11] typically operates within a single block, using *anchors* and *terminal points* to extend block-local cancellations; this increases implementation complexity and provides limited guarantees [39]. In contrast, SSA-based qubit-state renaming yields a simple correctness criterion—rotations merge *iff* they target the same SSA ID—thereby enabling global merging across block boundaries. Recent work by Amy and Lunderville [40] formulates rotation merging via relational program analysis, discovering additional opportunities across control flow and non-linear relations (Toffoli gates) for rotation gate merging. Our approach is complementary: it also exposes parity relationships between two-qubit gates, constructing a **Cross-block IR** that reveals long-range parity reuse and reduces CNOTs across blocks.

Before constructing rotation merging, we perform preprocessing to reduce redundant block barriers: (i) Propagate X gates forward via Clifford conjugation so that H gates remain the only block barriers; (ii) Cancel adjacent H pairs; (iii) If a CNOT gate is bracketed by H on both wires, cancel the four H gates and switch the control-target accordingly; if exactly one wire is bracketed, conjugate that wire and insert two H gates on the other wire to preserve equivalence. We interleave this preprocessing pass with rotation merging twice.

2) **Cross-block Parity Matrix Intermediate Representation Design**: We merge adjacent single-block phase polynomial regions into a larger phase polynomial block using a *cross-block IR*, as shown in Fig. 9(b). In the cross-block setting, post- H qubit states (new qubit rows) exist in the IR but remain *inactive* until their producer row (the original qubit wire before

the H) is eliminated and the H gate is inserted; only then is the row activated and available for operations.

To eliminate a pre- H row and activate its successor (e.g., q_3 in Fig. 9), three conditions must hold:

- 1) **No pending phase terms on the row:** the corresponding row in the phase-parity block is all zeros (no remaining phase terms depend on this state).
- 2) **Column isolation:** in the output-parity block, if the i -th row is to be removed, the i -th column must form a unit vector with its sole 1 located at row i (indicating that the target output state has been correctly prepared).
- 3) **Row isolation:** in the output-parity block, the i -th row must be a unit vector, containing a single 1 at its diagonal position in the output matrix.

When these are satisfied, this pre- H row can be removed, which activates the post- H row. The pre- H row maintains the correct output-parity, i.e., $q_1 \oplus q_2$ in Fig. 9(b), before it retires. Elimination of the pre- H row does not cause information loss: any correlations between the eliminated row and others have already been addressed or transferred. All subsequent transformations remain row operations induced by CNOTs.

3) *Linear Dependency Check for Correctness:* Once conditions (1) and (3) are satisfied, condition (2) follows directly. In such a case, the joint parity matrix contains a row of the form $v = [0 \cdots 0 \mid 0 \cdots 010 \cdots 0]$. Because all other entries of v are zero, adding this row to any other row clears the 1's in the same column, achieving column isolation. Therefore, we first make sure that the CNOT network synthesis process can produce circuit states that satisfy condition (3) together with condition (1), producing the required diagonal 1 in the output block, which is then used to enforce condition (2). Making sure these two conditions are satisfiable, therefore, is equivalent to checking whether the target row v can be written as an XOR (over $\text{GF}(2)$) of a set of candidate rows from the overall parity matrix. Equivalently, we must check whether the desired unit vector v lies in the span of these rows.

We use a rank-based test for this purpose. Let M be the matrix formed by the candidate rows. Appending v as an additional row, elimination of the pre- H row is feasible iff $\text{rank}(M \cup \{v\}) = \text{rank}(M)$, otherwise v is linearly independent of M and the elimination is impossible.

Hence, as we update the coupled matrix [phase-parity | output-parity] using CNOT gates, we constantly check whether the resulting parity matrix passes the rank test. If a state fails this rank test, it is immediately pruned. This check ensures that the pre- H row(s) are removable in the end, and significantly improves efficiency.

4) *Cross-Block Optimization Implementation:* Using the above representation and rank check, we integrate it with the co-optimization of the phase-parity network and the output-parity network (Section III-A). Post- H SSA rows remain *inactive* and are excluded from row-pair selection until their pre- H producer row satisfies conditions (1), (2), and (3). Then, we insert the H gate, and *activate* its post- H row.

a) *Feasibility and witness set:* We first run the linear dependency check (Section III-C3); if it is infeasible, we prune

the state. When feasible, solve $M\alpha = v$ over $\text{GF}(2)$ and let $S = \{i \mid \alpha_i = 1\}$ be a *witness set* whose XOR equals v . Write t for the index of the pre- H row to eliminate. If $t \notin S$, select any $j \in S$ and apply $\text{CNOT}(j, t)$. This replaces row_j by $\text{row}_j \oplus \text{row}_t$ without changing the span, yielding an equivalent witness set S' with $t \in S'$. *Row isolation (condition (3)).* For each $i \in S' \setminus \{t\}$, apply $\text{CNOT}(t, i)$ so that $\text{row}_i \leftarrow \bigoplus_{i \in S'} \text{row}_i = v$. *Column isolation (condition (2)).* For every $k \neq t$ with a 1 in column t , apply $\text{CNOT}(k, t)$ to clear that entry. *Eliminate and activate.* Remove row t and insert H , which activates the post- H SSA row. Fig. 10 illustrates a case with $t \notin S$: we first bring t into the witness with one CNOT, then isolate the row, clear the column, and finally eliminate t and insert H to activate the post- H state.

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{\text{CNOT}(q_1, q_0)} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{\begin{smallmatrix} \text{CNOT}(q_0, q_1) \\ \text{CNOT}(q_0, q_2) \\ \text{CNOT}(q_1, q_0) \end{smallmatrix}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{\begin{smallmatrix} \text{Eliminate } q_0 \\ \text{Insert } H \end{smallmatrix}} \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Fig. 10: Example of the $t \notin S$ case, $t = 1$ is the row to be eliminated, and S is the witness set which will XOR to the target row. (A) Initial cross-block intermediate representation, currently, the first row is expected to be eliminated, and the fourth row will be activated after that. (B) Bring t into the witness combination, yielding an equivalent witness set S' with $t \in S'$. (C) Isolate row/column t using CNOTs. (D) Eliminate t and insert H to activate the post- H state.

b) *Scalability and robustness:* The cross-block mechanism ensures correctness by locking inactive post- H rows and pruning infeasible states, but merging blocks increases complexity and may occasionally underperform the single-block optimization. To address this, we apply two strategies.

First, block merging is performed during a forward traversal of the block DAG obtained after applying the partition rules in Section III-C1. Each block corresponds to a node, with edges representing qubit dependencies. At each node, we evaluate merge opportunities with its immediate predecessor blocks.

To control complexity, we apply pruning: two blocks are merged only if they share multi-qubit interactions. The merging terminates when no further sharable interactions exist or when the merged block reaches the predefined size limit. For example, in Fig. 9, Block 1 acts on $\{q_0, q_1, q_2\}$ and Block 2 on $\{q_0, q_1, q_3\}$. Since they share $\{q_0, q_1\}$, merging is allowed.

Second, we adopt an *Incremental Block Merging* strategy: we begin with single-block optimization as a stable baseline, then progressively apply cross-block merging gradually and interleave with local refinement. If a merge fails to improve upon the previous, we revert it and keep the previous state.

IV. EVALUATION

We implement the proposed techniques in *PhasePoly* and evaluate them through the following research questions:

Q1: How does *PhasePoly* compare with existing phase polynomial optimization methods?

Q2: Why is *PhasePoly* necessary in general optimization?

Q3: Can *PhasePoly* capture long-range optimization opportunities at scale?

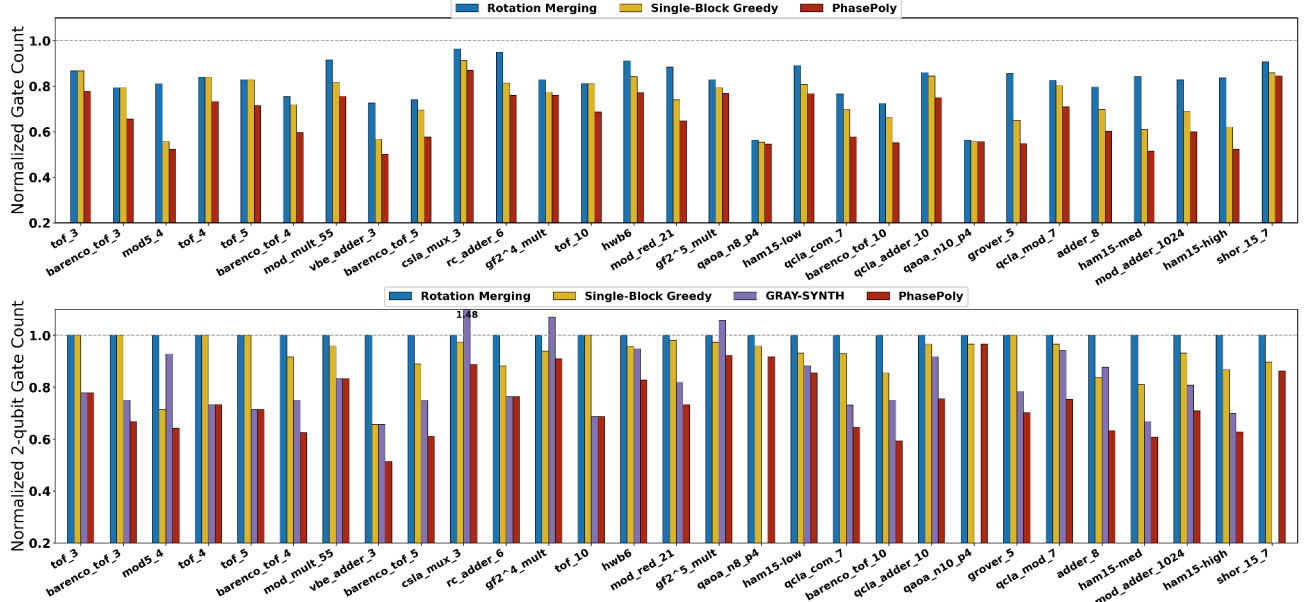


Fig. 11: Normalized total and two-qubit gate-count reductions across benchmark circuits, comparing *PhasePoly* against phase polynomial baselines. All values are normalized to the unoptimized circuits (1.0), with lower bars indicating greater reduction.

Q4: Do *PhasePoly*'s logical reductions translate to near-term hardware execution improvements?

Q5: How does *PhasePoly* benefit fault-tolerant compilation pipelines, and how should it be integrated??

Q6: Is the cross-block optimization correct and robust?

Q7: What are the compilation cost and parameter sensitivity of *PhasePoly*?

A. Experiment Setup

Benchmarks. In our experiment design, we selected a set of benchmark circuits that have been widely used in previous circuit optimization research [9]–[11], [17]–[19], [39], [41], [42], complemented by new benchmarks representing near-term and fault-tolerant applications. The suite includes quantum arithmetic circuits, MCX, Hamming coding functions, Hamiltonian simulation, QAOA, Grover's algorithm, and Shor's algorithm.

Baselines. We compare *PhasePoly* with three phase polynomial baselines: (i) *Rotation Merging* [11] merges phase gates that share identical phase polynomials, but does not optimize the CNOT network. (ii) *Single-block Greedy Optimization* [10], [15] greedily synthesizes the phase-parity network within each phase-polynomial block. The output-parity network is handled separately using Gaussian elimination. (iii) Gray-Synth [10] reduces two-qubit gates using the *sum-over-paths* representation. We report the CNOT-count results from the original paper.

We also integrate our technique into two general-purpose optimizers. We use Quartz [17] and QUESO [18]. Quartz performs equivalent **subcircuit rewriting** and does not model phase contributions. QUESO adopts the *sum-over-paths* form to enhance phase polynomial ECCs and uses search-based rewriting methods to enlarge optimization coverage.

Setup. All experiments run on a 2.8 GHz AMD EPYC 7313 CPU. *PhasePoly* uses a priority queue and a solution pool

with maximum sizes chosen according to the runtime budget. We enable an *Incremental Block Merging* strategy to improve robustness during cross-block optimization. For Quartz and QUESO, we follow their recommended equivalent subcircuit sizes and allocate up to 2-hour per circuit. *PhasePoly* attains the reported results without consuming the full runtime budget.

B. Q1: Comparison with Phase Polynomial Baselines

We compare *PhasePoly* with three phase polynomial baselines, as summarized in Fig. 11. *Rotation Merging* combines only rotation gates with identical phase polynomials and leaves the CNOT network unchanged, which limits the overall optimization gains. We implement this pass using our *SSA-style rotation-merging* infrastructure. *Single-block Greedy Optimization* is reproduced in our infrastructure as an independent per-block optimization pass, where the phase-parity and output-parity networks are synthesized separately using greedy synthesis and Gaussian elimination, respectively, without cross-block optimization. This baseline reduces total gates by 26.93% and two-qubit gates by 8.14% on average. Gray-Synth [10], built on T-par [9], targets CNOT reduction; its reported results show an average CNOT reduction of 17.62%.

In contrast, *PhasePoly* co-optimizes **the phase-parity network and the output-parity network** and employs *cross-block IR and optimization*. It achieves up to 50% total-gate reduction and 48.57% CNOT reduction—**34.70%** and **26.83%** on average—surpassing all baselines and improving upon Gray-Synth by 9.21% in CNOT reduction.

Q1 Summary: *PhasePoly* outperforms by jointly optimizing phase- and output-parity and exploiting cross-block opportunities missed by phase-only, single-block methods.

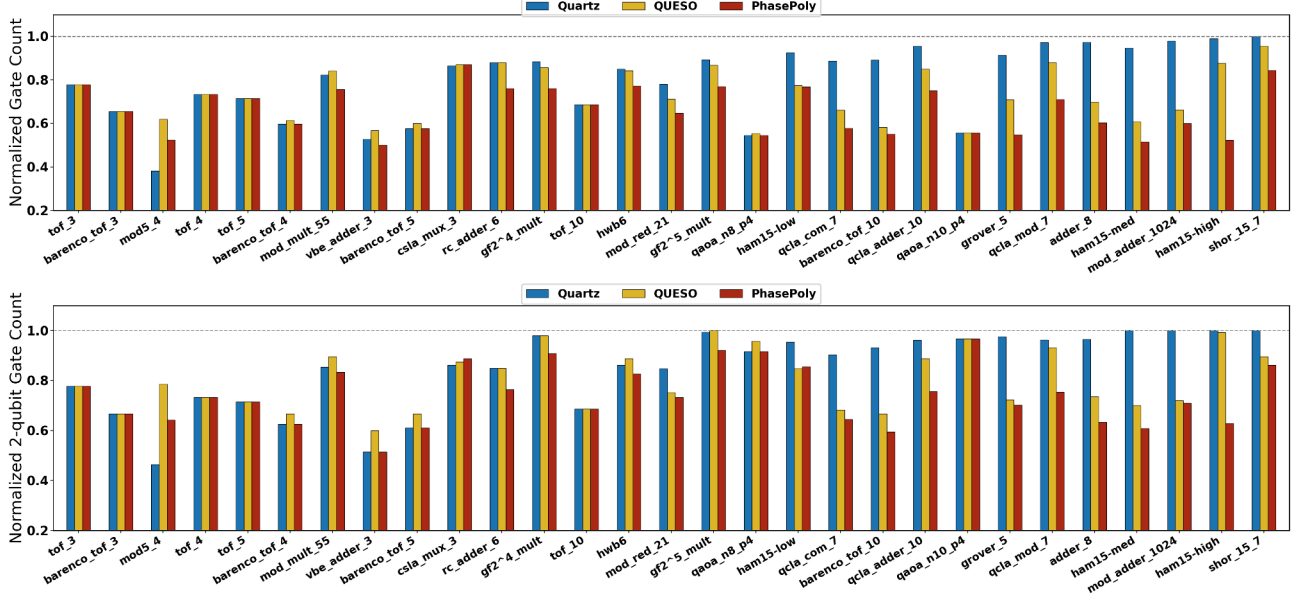


Fig. 12: Normalized total and two-qubit gate-count reductions across benchmark circuits, comparing *PhasePoly* against general circuit optimizers. All values are normalized to the unoptimized circuits (1.0), with lower bars indicating greater reduction.

C. Q2: Necessity in General Circuit Optimization

PhasePoly is orthogonal to subcircuit rewriting frameworks and can be composed with existing optimization passes. To study their interaction, we integrate *PhasePoly* with two state-of-the-art subcircuit rewriting frameworks—Quartz [17] and QUESO [18]—and evaluate them under their recommended settings (3-qubit / 6-gate subcircuits, 2-hour per circuit). Fig. 12 summarizes their standalone performance; note that this is not an apples-to-apples comparison: although they optimize gates beyond phase polynomial structure, *PhasePoly* still delivers the strongest reductions: Quartz and QUESO reduce total gate by 22.17% and 27.83% (CNOTs by 16.88% and 20.70%), while *PhasePoly* achieves total reduction by 34.70% (CNOTs by 26.83%) on average.

a) *Effect of circuit scale*: On average, *PhasePoly* achieves greater reductions in both total and CNOT gates, though it is not always the best on small circuits. To analyze this trend, we group benchmarks by original gate count:

Small (<200 gates): Taking the best of Quartz and QUESO per circuit, *subcircuit rewriting* ties *PhasePoly* on 6 circuits and surpasses it on 2 of 10.

Medium (200–500 gates): Only 3 circuits tie or exceed *PhasePoly* among 10.

Large (>500 gates): Only one QAOA circuit ties *PhasePoly*; all other circuits exhibit significant performance gaps.

These results are consistent with design intent: *PhasePoly* leverages the *phase polynomial* and *cross-block* IR to capture long-range structure across the circuit, while subcircuit rewriting—bounded by local equivalence patterns—loses effectiveness as it scales up despite covering more gate types.

b) *Integrating optimization passes*: Fig. 13 analyzes how *PhasePoly* interacts with subcircuit rewriting in a combined pipeline. We denote sequential application as “A+B” (run A

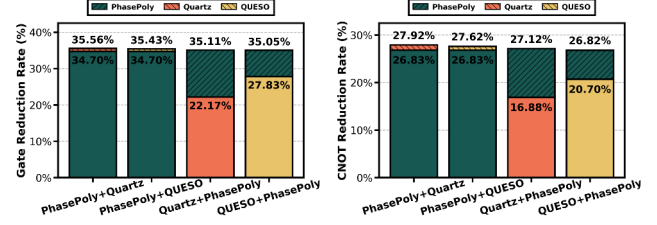


Fig. 13: Integration of *PhasePoly* with Quartz and QUESO. Bars show average total-gate (left) and CNOT (right) reduction rates relative to original circuits. “A+B” applies A then B; solid and hatched bars indicate first and second passes.

first, then B). Applying subcircuit rewriting after *PhasePoly* yields modest additional gains (≈ 0.75 – 1.25% in total and CNOT reductions). Applying *PhasePoly* after rewriting provides substantially larger improvements (≈ 6 – 13%), revealing that *PhasePoly* identifies long-range opportunities left unexploited by local subcircuit rewriting.

Although QUESO slightly outperforms Quartz as a standalone pass—thanks to its phase modeling—both benefit when combined with *PhasePoly*. Quartz + *PhasePoly* performs better than QUESO + *PhasePoly*. Quartz does not consider any phase information, and therefore mainly removes local redundancy. This implies the benefit of having a dedicated phase polynomial pass, rather than having the phase polynomial optimization spread into different optimization passes.

Q2 Summary: *PhasePoly* complements local subcircuit rewriting: the former enables long-range CNOT/phase reductions, while the latter is effective on small circuits. Together, they unlock optimizations neither achieves alone.

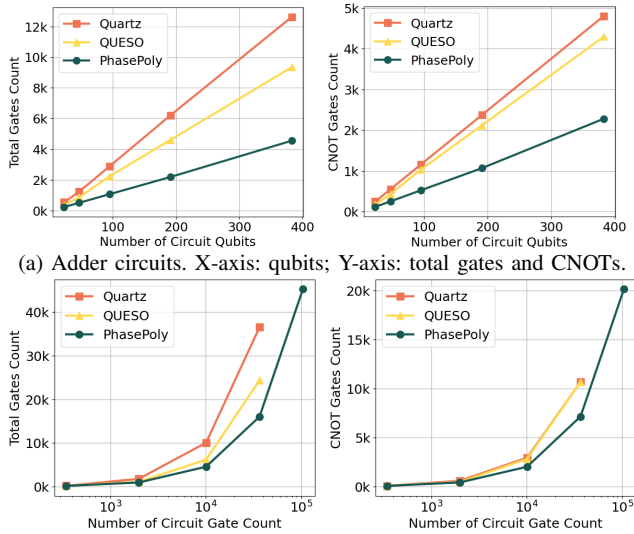
D. Q3: Can PhasePoly Effectively Scale to Large Circuits?

Q2 shows that the gap between *PhasePoly* and subcircuit rewriting widens as circuits grow. Search-based *subcircuit rewriting* enumerates many candidate equivalent subcircuits before applying a rule. As the target pattern size grows, the search space explodes exponentially, so practical deployments restrict patterns to limited windows. This locality makes it difficult to realize long-range optimization opportunities that span large portions of a circuit.

We further stress-test scalability on three large-circuit families: (i) *MCX* (multi-controlled-NOT) circuits [20], where qubit and gate counts grow roughly linearly (19–499 qubits; 480–14,880 gates); (ii) *Adder* circuits (23–383 qubits; 637–12,637 gates); and (iii) *HWB* (Hamming coding functions) [29] with a fixed count of 16 qubits but rapidly growing gates (345–104,068). We compare *PhasePoly* with Quartz and QUESO under a 2-hour time budget; *PhasePoly* never exceeded 5,500 seconds even on the largest instance (hwb8_113).

Fig. 14 reports total-gate and CNOT reductions for adder and HWB circuits, complementing the MCX results in Fig. 5. As circuit size increases, Quartz and QUESO saturate or fail on the largest instances, whereas *PhasePoly* continues to achieve substantial reductions. These results show that structured parity-matrix reasoning and cross-block optimization expose long-range opportunities that fixed-window subcircuit rewriting struggles to capture.

Q3 Summary: On large circuits, *PhasePoly* sustains and widens its advantage by leveraging its global parity reasoning and cross-block optimization, capturing long-range reductions that *subcircuit rewriting* cannot.



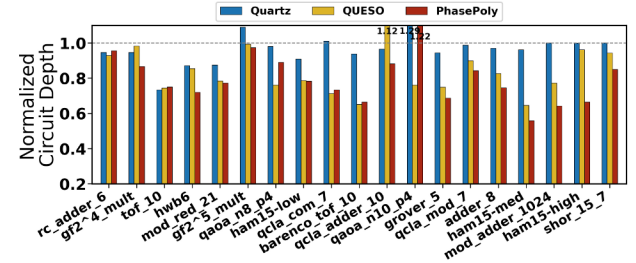
(b) HWB circuits. X-axis: original gate count (log scale); Y-axis: total gates and CNOTs. Missing points denote optimization failures.

Fig. 14: Comparison of large circuits, *PhasePoly* scales effectively, while subcircuit rewriting saturates on large circuits.

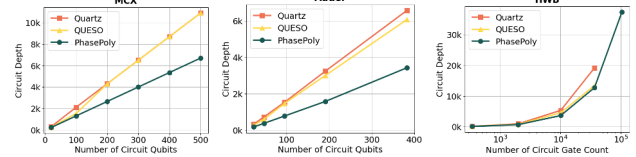
E. Q4: Hardware-Level Execution Improvements

On near-term hardware, execution quality is often limited by (i) circuit depth (accumulated error) and (ii) constrained connectivity, where non-local two-qubit interactions introduce routing overhead (SWAPs). Therefore, gate-count reductions must be validated against depth and physical circuit metrics.

Logical depth. Fig. 15(a) reports normalized logical circuit depth (excluding small-size circuits for readability). On average, Quartz reduces depth by 13.26%, QUESO by 19.64%, and *PhasePoly* achieves the largest reduction of 22.47%, corresponding to a 1.14–1.69 $\times$ improvement. The gap widens on large circuit families in Fig. 15(b), where Quartz reduces depth by 4.03%, QUESO by 14.45%, and *PhasePoly* by 40.91%, corresponding to a 2.83–10.15 $\times$ improvement.



(a) Normalized circuit depth reductions across benchmark circuits.



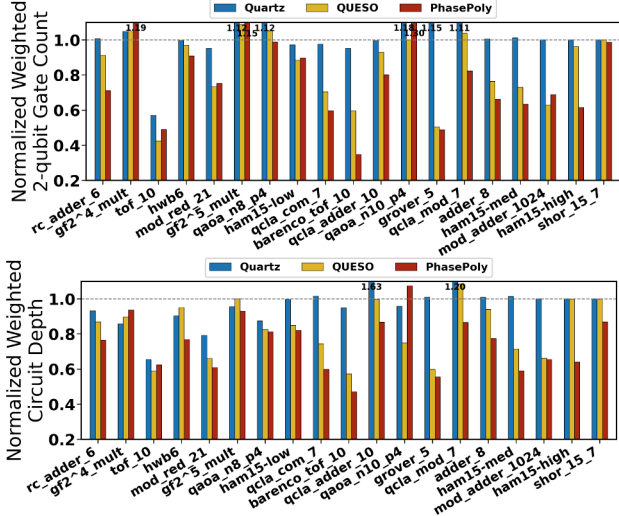
(b) Normalized circuit depth reductions for MCX, Adder, and HWB circuit families. The HWB x-axis is shown in log scale.

Fig. 15: Normalized logical circuit depth reductions across benchmark circuits and three large circuit families.

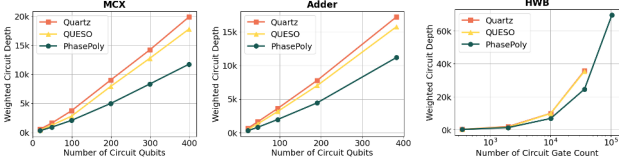
Hardware mapping under limited connectivity. To test whether logical improvements persist after routing, we map optimized circuits to 2D planar coupling graphs (square grids) and perform routing using Qiskit SABRE [6], [43], [44]. We report (i) *weighted two-qubit gate count* and (ii) *physical circuit depth*, where each SWAP is weighted as three CNOTs.

Fig. 16(a) shows that, across benchmark circuits, on average, Quartz reduces physical circuit depth by 15.23%, QUESO by 21.60%, while *PhasePoly* achieves the largest reductions of 28.35%, corresponding to a 1.31–1.86 $\times$ improvement in physical circuit depth. On large circuit families (Fig. 16(b)), the advantage becomes stronger: Quartz reduces physical depth by 2.7%, QUESO by 15.25%, and *PhasePoly* by 40.84%, corresponding to a 2.68–15.13 $\times$ improvement.

Why hardware mapping can amplify circuit optimization gains (and when it does not). In many cases, *PhasePoly*'s logical reductions *amplify* after mapping (e.g., depth reductions of 22.47% become 28.35% in physical circuits), because fewer two-qubit interactions lead to fewer SWAPs. Cross-block optimization further helps by reusing parity structures across phase-polynomial blocks, reducing repeated reconstruc-



(a) Normalized weighted two-qubit gate count and physical circuit depth across benchmark circuits.



(b) Weighted physical circuit depth for MCX, Adder, and HWB circuit families. The HWB x-axis is shown in log scale.

Fig. 16: Reductions in weighted physical circuit 2-qubit gate-count and depth after hardware mapping.

tion of multi-qubit interactions. *PhasePoly* matches or outperforms Quartz and QUESO on most benchmarks. However, for `qaoa_n10_p4`, CNOT count decreases by $\sim 3\%$ but depth increases by $\sim 20\%$ due to gate-count optimization, potentially reducing parallelism. For QAOA-type circuits, domain-specific hardware mappers [45], [46] exist and exploit commuting flexibilities unique to QAOA circuits. We expect that the domain-specific hardware mappers may further reduce gate counts on top of already optimized logical circuits.

For circuits that are already topology-friendly (e.g., `gf2^4_mult`, `gf2^5_mult`), routing overhead can dominate and offset logical gate-count gains (all optimized circuits have worse physical performance than original circuits). Overall, for the majority of cases, as circuits scale, *PhasePoly*'s reductions translate reliably into better physical circuit depth and two-qubit gate cost.

Q4 Summary: *PhasePoly* reduces not only gate counts but also logical depth. Under constrained connectivity, these gains persist—and often amplify—after routing, yielding substantial reductions in depth and two-qubit gate cost, especially for large circuits.

F. Q5: Fault-Tolerant Compilation Benefits and Integration

Fault-tolerant quantum computing (FTQC) is limited by qubit overhead, runtime, and architectural constraints. Many logical/NISQ optimizations extend to FT compilation [47].

Traditional FT analyses focus on T -gate count/depth due to the high cost of magic-state distillation [24]. Recent advances in magic-state cultivation [25]–[27] substantially reduce this overhead, making Clifford costs increasingly important. Modern cost models suggest that CNOTs can be **comparable** in spacetime cost to T states of similar reliability [25], with **non-constant** ancilla volume and operation depth [28]. Since *PhasePoly* reduces both CNOT and R_z structure at the logical level, it can improve downstream FT resource costs.

FT resource estimation. Using the Azure Resource Estimator [48], [49] under a surface-code, nearest-neighbour architecture [50], [51], we perform end-to-end FT resource estimation. Fig. 17 reports normalized wall-clock runtime relative to unoptimized circuits, excluding small-size and parameterized circuits. Quartz, QUESO, and *PhasePoly* achieve average reductions of 11.99%, 31.80%, and 44.62%, respectively, with *PhasePoly* providing the largest improvement.

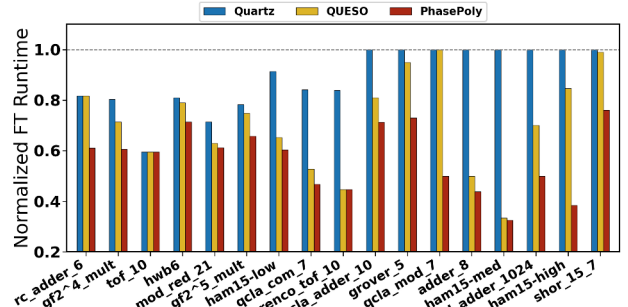


Fig. 17: Normalized fault-tolerant circuit wall-clock runtime.

Integration with Clifford+ T synthesis. Each arbitrary R_z rotation must be synthesized into an FT instruction set such as Clifford+ T (H , S , and T sequence). We therefore study how *PhasePoly* interacts with FT gate synthesis by combining it with GridSynth [52] on 14 variational circuits: QAOA Max-Cut on 3-regular graphs (4–24 qubits, 2,150–12,900 gates) and VQE circuits, including UCCSD ansatz with Jordan-Wigner (JW) [53], Bravyi-Kitaev (BK) [54], and parity (P) [55] encodings, as well as the Hamming-weight-preserving ansatz (HW) [56] (4–12 qubits, 2,641–231,780 gates). We compare two compilation orders: (A) **GridSynth** $\rightarrow$ *PhasePoly* and (B) *PhasePoly* $\rightarrow$ **GridSynth**. The purpose of investigating this is to see how to better apply *PhasePoly* into the overall compilation pipeline. Both pipelines apply the same commuting-rule simplification as the final pass.

Fig. 18 reports normalized T count, two-qubit gate count, and circuit depth. Running *PhasePoly* before GridSynth produces the lowest depth on most circuits because large $\{\text{CNOT}, R_z\}$ regions are simplified before GridSynth introduces additional H gates that split phase-polynomial blocks and limit rotation-merging opportunities.

Across benchmarks, we observe: (i) **T -count changes are modest**, noticeable mainly for HWP circuits; (ii) **two-qubit gate reductions are common** using *PhasePoly*'s optimization and often translate to lower depth; and (iii) **the benefit**

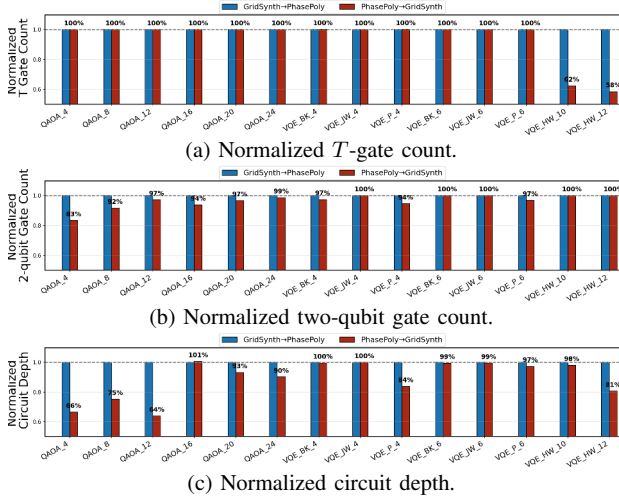


Fig. 18: Comparison of two compilation orders. Metrics are normalized to GridSynth $\rightarrow$ PhasePoly.

depends on the regularity of circuit structure: circuits with structured phase interactions (e.g., parity and HWP ansatz) obtain larger improvements ($\sim 10\%$ depth reduction on average), while JW/BK encodings show $< 1\%$ change due to their already compact CNOT- R_z structure.

Q5 Summary: PhasePoly’s strong CNOT/ R_z reductions, long-range optimization, and natural fit for Clifford+ T circuits make it effective for fault-tolerant compilation. PhasePoly is most effective when applied before Clifford+ T synthesis, which introduces additional H barriers.

G. Q6: Correctness and Robustness of PhasePoly

a) *Equivalence checking for correctness:* Phase polynomials faithfully model $\{\text{CNOT}, R_z\}$ circuits and are widely used for verification [12], [13], [18]. Because PhasePoly introduces *cross-block IR and optimization* that merges multiple phase-polynomial blocks, we enforce additional constraints and pruning rules to guarantee that all intermediate states remain legal and synthesizable (Section III-C3).

We also perform end-to-end equivalence checking: for circuits with fewer than 8 qubits, we compare unitaries using Qiskit [6], and for all circuits we use the formal verification tool MQT QCEC [57]. All checked benchmarks pass verification; `mod_adder_1024` is excluded because it exceeds our hardware limits.

b) *Incremental block merging for robustness:* Cross-block optimization can yield additional reductions—about one third of benchmarks (9 circuits) benefit from it. We use *incremental block merging* that expands the merge size gradually, optimizing step by step rather than merging all blocks at once.

Table I evaluates three representative circuits under the same parameters except for the merge size (“Group k ”, merging 1-7 adjacent blocks) and our *Incremental* strategy, which gradually increases k and keeps only beneficial steps. For `barenco_tof_10`, improvements emerge at Group 3 and stabilize, matching the *Incremental* result. For `adder_8`,

Circuit	barenco_tof_10		adder_8		ham15_med	
	# Gates	# CXs	# Gates	# CXs	# Gates	# CXs
Org.	450	192	900	409	1272	534
Group 1	262	128	557	274	696	353
Group 3	248	114	542	259	695	352
Group 5	248	114	540	257	693	350
Group 7	248	114	540	257	694	351
Incremental	248	114	542	259	656	325

TABLE I: Effect of cross-block merge size on optimization quality for three typical circuits. “Group k ” merges k adjacent blocks at a time, $k \in \{1, 3, 5, 7\}$; “Incremental” increases k stepwise up to 7, retaining gains and pruning regressions. **Bold** numbers denote the best value in each column.

larger groups continue to help (CX: 274 $\rightarrow$ 257), while *Incremental* remains close (259). For `ham15_med`, performance peaks at Group 5 (353 $\rightarrow$ 350) but degrades at Group 7; *Incremental* avoids this and achieves the best (325). Overall, *Incremental Block Merging* offers a robust approach that captures large gains while avoiding over-merging regressions.

Q6 Summary: Our cross-block IR and optimization preserve correctness (all verified except one timeout) and *Incremental block merging* strategy yields robust gains while avoiding over-merging side effects.

H. Q7: Compilation Cost and Parameter Sensitivity

PhasePoly expands the optimization space beyond greedy phase-parity synthesis by jointly optimizing phase and output parities and enabling cross-block merging. We therefore evaluate two practical questions: how quickly it converges, and how sensitive it is to the search parameters.

Compilation time. Fig. 19 shows optimization progress under increasing runtime budgets using a deliberate over-optimization *Incremental Block Merging* strategy. PhasePoly reaches 32.37% average gate reduction within 1,200 s, and 86.21% of benchmarks stabilize by 1,562 s. Reductions further improve to 33.35% at 2,400 s and converge around 34.69% by 3,600 s. The slowest case, `ham15-high`, finishes in 5,025 s, still below the 7,200 s budget used for search-based subcircuit rewriting, while achieving substantially larger reductions.

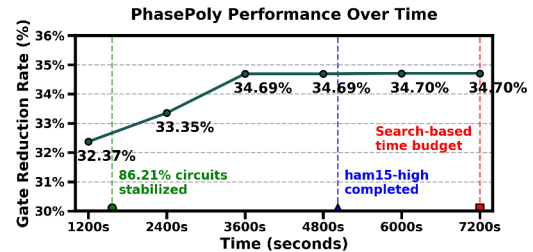


Fig. 19: Optimization progress of PhasePoly over time. Average reduction reaches 32.37% at 1,200 s and converges near 34.7% by 3,600–4,800 s.

Parameter sensitivity. We evaluate the sensitivity of PhasePoly’s search parameters to: (1) priority-queue bound Q , (2) solution-pool size P , and (3) cross-block group size G . We denote settings as (Q, P, G) and test the five largest circuits in our benchmark suite (12–28 qubits, 900–36,598 gates).

Queue/pool sizes: diminishing returns. With cross-block disabled ($G=1$), we vary Q and P from 1 to 20,000 separately.

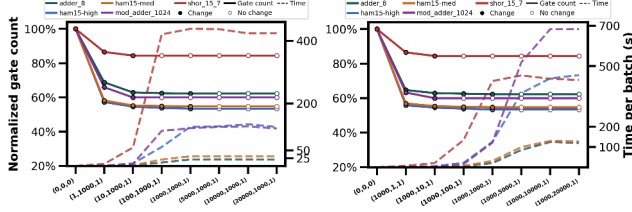


Fig. 20: Sensitivity to Q and P with $G=1$ (no cross-block optimization). Each x-tick is (Q, P, G) . Left y-axis: normalized gate count; right y-axis: time needed (seconds). Solid: normalized reduction; dashed: runtime. Filled markers: improved quality; hollow markers: no change. Quality improves initially, then quickly saturates, while runtime continues to increase, especially with larger P .

(i) When Q is fixed at 1000, increasing P improves quality only up to a moderate bound ($P \in [100, 1000]$), after which reductions plateau while runtime increases (Fig. 20 right). Similarly, with $P=1000$, quality also saturates at $Q \in [100, 1000]$ (Fig. 20 left). After that, although we increase the queue size to 20,000, the runtime grows slowly, which indicates that moderate settings ($Q \in [100, 1000]$) are sufficient to reliably discover the top ~ 1000 candidate solutions without requiring a larger search space.

Joint parameter scaling and feasibility under cross-block optimization. Jointly scaling (Q, P, G) confirms the same robustness trend. Fig. 21 shows representative results for $G \in \{3, 7\}$; results for $G \in \{1, 5\}$ follow the same pattern and are omitted for readability. Across all group sizes, reductions saturate near $Q=P=1000$, while larger bounds provide only marginal quality improvement at substantially higher runtime. For example, increasing the solution pool from 1,000 to 20,000 increases runtime by nearly $20\times$ but yields only marginal additional reductions for ham15-high and mod_adder_1024 across different group-size settings. When the bounds are extremely tight ($Q=P=1$), the bounded search may discard states needed to satisfy the rank-based correctness constraints, causing rare optimization failures; we observed this for ham15-med with $G \in \{5, 7\}$.

Q7 Summary: *PhasePoly* converges under moderate runtime budgets even under a deliberate over-optimization incremental block merging strategy. It is also robust to search parameters: moderate bounds ($Q=P=1000$) consistently achieve near-optimal reductions, while larger search spaces mainly increase compilation time with diminishing returns.

V. RELATED WORK

Manual Rule-based Optimization. Many optimizers rely on manually written, equivalence-preserving rewrite rules [6]–[8], [58]. To ensure correctness, verification-oriented compilers and optimizers have been developed [12], [39], [57], [59]–[61], guaranteeing each applied rule is equivalence-preserving.

Search-Based Subcircuit Rewriting. Many optimizers rely on rule-based *subcircuit rewriting*, where small patterns are matched and replaced. Systems such as Quanto [62],

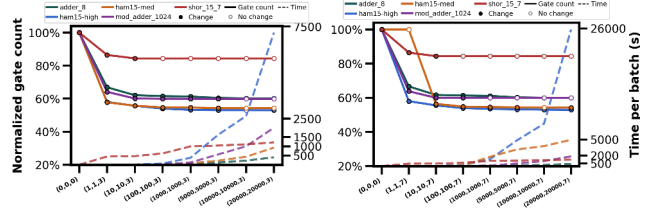


Fig. 21: Joint scaling of (Q, P, G) for $G \in \{3, 7\}$. Reductions saturate at moderate bounds (e.g., $Q=P=1000$), while runtime grows rapidly for larger bounds. Extremely tight bounds can miss feasible cross-block solutions in rare cases. When $G \in \{1, 5\}$, the trend is also consistent.

Quartz [17], and QUESO [18] generate such rules automatically and apply them through global search, but their patterns are usually limited to small 3-qubit/6-gate regions, restricting long-range improvements. Reinforcement-learning approaches [42], [63], [64] explore larger spaces but still depend on fixed rule sets and require costly pretraining.

Phase Polynomial Optimization. Prior work typically optimizes only the *phase-parity* network [10], [14], [15], [65], leaving the *output-parity* network to other passes. Nam *et al.* [11] consider both but focus on random floating and merging of rotation gates. Other phase polynomial methods [9], [19], [66]–[68] mainly target T -count (often combined with higher-level techniques such as tensor-rank decomposition) rather than full CNOT/ R_z optimization.

Unitary Synthesis and Hamiltonian Decomposition. Unitary-synthesis approaches [69]–[77] optimize programs by synthesizing circuits for target unitaries. However, they often rely on approximate equivalence, requiring explicit error budgeting, and their scalability is limited. Domain-specific decompositions, such as those for Hamiltonian simulation [78]–[83], achieve application- and hardware-driven improvements rather than general-purpose, exactly equivalent circuit optimization.

VI. CONCLUSION

We revisit *phase polynomial* optimization with *PhasePoly*, which jointly optimizes the *phase-parity* and *output-parity* networks using a cross-block intermediate representation to enable long-range optimizations beyond single blocks. We advocate making phase polynomial optimization a standard component of the compilation pipeline.

ACKNOWLEDGMENTS

The authors thank Zirui Li, Minghao Guo, Jiakang Li, and Caitlin Chan (Rutgers University), Hanyu Wang (UCLA), and Mu-Te Lau (Northwestern University) for helpful discussions and feedback. This work was funded by Rutgers Research Council, the National Science Foundation (NSF), and the U.S. Department of Energy (DOE). In particular, Z.C., H.C., V.C., and E.Z. were supported by DOE Award DE-SC0025563, NSF Award CCF-2129872, NSF Award CCF-2529338, and a Rutgers Research Council Grant. This work was also funded by the National Research Foundation of Korea (NRF) under the project, “Creation of the Quantum Information Science R&D Ecosystem Based on Human Resource” (RS-2023-00303229).

REFERENCES

- [1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” *Proceedings - Annual IEEE Symposium on Foundations of Computer Science, FOCS*, 1994.
- [2] J. Proos and C. Zalka, “Shor’s discrete logarithm quantum algorithm for elliptic curves,” *arXiv preprint quant-ph/0301141*, 2003.
- [3] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC ’96. New York, NY, USA: Association for Computing Machinery, 1996, p. 212–219. [Online]. Available: <https://doi.org/10.1145/237814.237866>
- [4] R. P. Feynman, “Simulating physics with computers,” in *Feynman and computation*. cRc Press, 2018, pp. 133–153.
- [5] Y. Cao, J. Romero, J. P. Olson, M. Degroote, P. D. Johnson, M. Kieferová, I. D. Kivlichan, T. Menke, B. Peropadre, N. P. Sawaya *et al.*, “Quantum chemistry in the age of quantum computing,” *Chemical reviews*, vol. 119, no. 19, pp. 10856–10915, 2019.
- [6] G. Aleksandrowicz, T. Alexander, P. Barkoutsos, L. Bello, Y. Ben-Haim, D. Bucher, F. J. Cabrera-Hernández, J. Carballo-Franquis, A. Chen, C.-F. Chen, J. M. Chow, A. D. Córcoles-Gonzales, A. J. Cross, A. Cross, J. Cruz-Benito, C. Culver, S. D. L. P. González, E. D. L. Torre, D. Ding, E. Dumitrescu, I. Duran, P. Eendebak, M. Everitt, I. F. Sertage, A. Frisch, A. Fuhrer, J. Gambetta, B. G. Gago, J. Gomez-Mosquera, D. Greenberg, I. Hamamura, V. Havlicek, J. Hellmers, Łukasz Herok, H. Horii, S. Hu, T. Imamichi, T. Itoko, A. Javadi-Abhari, N. Kanazawa, A. Karazeev, K. Krsulich, P. Liu, Y. Luh, Y. Maeng, M. Marques, F. J. Martín-Fernández, D. T. McClure, D. McKay, S. Meesala, A. Mezzacapo, N. Moll, D. M. Rodríguez, G. Nannicini, P. Nation, P. Ollitrault, L. J. O’Riordan, H. Paik, J. Pérez, A. Phan, M. Pistoia, V. Prutyanov, M. Reuter, J. Rice, A. R. Davila, R. H. P. Rudy, M. Ryu, N. Sathaye, C. Schnabel, E. Schoute, K. Setia, Y. Shi, A. Silva, Y. Siraichi, S. Sivarajah, J. A. Smolin, M. Soeken, H. Takahashi, I. Tavernelli, C. Taylor, P. Taylour, K. Trabing, M. Treinish, W. Turner, D. Vogt-Lee, C. Vuillot, J. A. Wildstrom, J. Wilson, E. Winston, C. Wood, S. Wood, S. Wörner, I. Y. Akhalwaya, and C. Zoufal, “Qiskit: An Open-source Framework for Quantum Computing,” Feb. 2019. [Online]. Available: <https://doi.org/10.5281/zenodo.2562111>
- [7] R. S. Smith, E. C. Peterson, M. G. Skilbeck, and E. J. Davis, “An open-source, industrial-strength optimizing compiler for quantum programs,” *Quantum Science and Technology*, vol. 5, no. 4, p. 044001, 2020.
- [8] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, “*t|ket*: a retargetable compiler for nisq devices,” *Quantum Science and Technology*, vol. 6, no. 1, p. 014003, 2020.
- [9] M. Amy, D. Maslov, and M. Mosca, “Polynomial-time t-depth optimization of clifford+ t circuits via matroid partitioning,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 10, pp. 1476–1489, 2014.
- [10] M. Amy, P. Azimzadeh, and M. Mosca, “On the controlled-not complexity of controlled-not-phase circuits,” *Quantum Science and Technology*, vol. 4, no. 1, p. 015002, 2018.
- [11] Y. Nam, N. J. Ross, Y. Su, A. M. Childs, and D. Maslov, “Automated optimization of large quantum circuits with continuous parameters,” *npj Quantum Information*, vol. 4, no. 1, p. 23, 2018.
- [12] M. Amy, M. Roetteler, and K. M. Svore, “Verified compilation of space-efficient reversible circuits,” in *International Conference on Computer Aided Verification*. Springer, 2017, pp. 3–21.
- [13] M. Amy, “Towards large-scale functional verification of universal quantum circuits,” *arXiv preprint arXiv:1805.06908*, 2018.
- [14] A. M.-v. de Griend and R. Duncan, “Architecture-aware synthesis of phase polynomials for nisq devices,” *arXiv preprint arXiv:2004.06052*, 2020.
- [15] V. Vandaele, S. Martiel, and T. G. de Brugière, “Phase polynomials synthesis algorithms for nisq architectures and beyond,” *Quantum Science and Technology*, vol. 7, no. 4, p. 045027, 2022.
- [16] X. Li, J. Liu, S. Xu, P. Hovland, and V. Chaudhary, “Hopps: Hardware-aware optimal phase polynomial synthesis with blockwise optimization for quantum circuits,” *arXiv preprint arXiv:2511.18770*, 2025.
- [17] M. Xu, Z. Li, O. Padon, S. Lin, J. Pointing, A. Hirth, H. Ma, J. Palsberg, A. Aiken, U. A. Acar *et al.*, “Quartz: superoptimization of quantum circuits,” in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022, pp. 625–640.
- [18] A. Xu, A. Molavi, L. Pick, S. Tannu, and A. Albarghouthi, “Synthesizing quantum-circuit optimizers,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 835–859, 2023.
- [19] F. J. Ruiz, T. Laakkonen, J. Bausch, M. Balog, M. Barekatain, F. J. Heras, A. Novikov, N. Fitzpatrick, B. Romera-Paredes, J. Van De Wetering *et al.*, “Quantum circuit optimization with alphasensor,” *Nature Machine Intelligence*, vol. 7, no. 3, pp. 374–385, 2025.
- [20] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, “Elementary gates for quantum computation,” *Physical review A*, vol. 52, no. 5, p. 3457, 1995.
- [21] R. Iten, R. Colbeck, I. Kukuljan, J. Home, and M. Christandl, “Quantum circuits for isometries,” *Physical Review A*, vol. 93, no. 3, p. 032318, 2016.
- [22] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [23] E. Jang, D. Ha, S. Choi, Y. Kim, J. Kwon, Y. Lee, S. Ahn, H. Kim, and W. W. Ro, “Recompiling qaoa circuits on various rotational directions,” in *Proceedings of the 2024 international conference on parallel architectures and compilation techniques*, 2024, pp. 309–324.
- [24] S. Bravyi and A. Kitaev, “Universal quantum computation with ideal clifford gates and noisy ancillas,” *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 71, no. 2, p. 022316, 2005.
- [25] C. Gidney, N. Shutty, and C. Jones, “Magic state cultivation: growing t states as cheap as cnot gates,” *arXiv preprint arXiv:2409.17595*, 2024.
- [26] Z.-H. Chen, M.-C. Chen, C.-Y. Lu, and J.-W. Pan, “Efficient magic state cultivation on $\mathbb{R}P^2$,” *arXiv preprint arXiv:2503.18657*, 2025.
- [27] E. Rosenfeld, C. Gidney, G. Roberts, A. Morvan, N. Lacroix, D. Kafri, J. Marshall, M. Li, V. Sivak, D. Abanin *et al.*, “Magic state cultivation on a superconducting quantum processor,” *arXiv preprint arXiv:2512.13908*, 2025.
- [28] W. J. Huggins, T. Khattar, A. Xu, M. Harrigan, C. Kang, G. H. Low, A. Fowler, N. C. Rubin, and R. Babbush, “The fluid allocation of surface code qubits (flasq) cost model for early fault-tolerant quantum algorithms,” *arXiv preprint arXiv:2511.08508*, 2025.
- [29] M. Saeedi, M. S. Zamani, M. Sedighi, and Z. Sasanian, “Reversible circuit synthesis using a cycle-based approach,” *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 6, no. 4, pp. 1–26, 2010.
- [30] K. N. Patel, I. L. Markov, and J. P. Hayes, “Optimal synthesis of linear reversible circuits,” *Quantum Inf. Comput.*, vol. 8, no. 3, pp. 282–294, 2008.
- [31] T. G. De Brugière, M. Baboulin, B. Valiron, S. Martiel, and C. Allouche, “Gaussian elimination versus greedy methods for the synthesis of linear reversible circuits,” *ACM Transactions on Quantum Computing*, vol. 2, no. 3, pp. 1–26, 2021.
- [32] B. Zindorf and S. Bose, “Efficient implementation of multi-controlled quantum gates,” *arXiv preprint arXiv:2404.02279*, 2024.
- [33] J. M. Arrazola, O. Di Matteo, N. Quesada, S. Jahangiri, A. Delgado, and N. Killoran, “Universal quantum circuits for quantum chemistry,” *Quantum*, vol. 6, p. 742, 2022.
- [34] J. van de Wetering and M. Amy, “Optimising quantum circuits is generally hard,” *arXiv preprint arXiv:2310.05958*, 2023.
- [35] S. Russell, “Efficient memory-bounded search methods,” in *ECAI*, vol. 92, 1992, pp. 1–5.
- [36] H. Kaïndl and A. Khorsand, “Memory-bounded bidirectional search,” in *AAAI*, 1994, pp. 1359–1364.
- [37] D. Gottesman, “Theory of fault-tolerant quantum computation,” *Physical Review A*, vol. 57, no. 1, p. 127, 1998.
- [38] F. Rastello and F. B. Tichadou, *SSA-based compiler design*. Springer, 2022.
- [39] K. Hietala, R. Rand, S.-H. Hung, X. Wu, and M. Hicks, “A verified optimizer for quantum circuits,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–29, 2021.
- [40] M. Amy and J. Lunderville, “Linear and non-linear relational analyses for quantum program optimization,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, pp. 1072–1103, 2025.
- [41] A. Kissinger and J. van de Wetering, “Pyzx: Large scale automated diagrammatic reasoning,” *arXiv preprint arXiv:1904.04735*, 2019.
- [42] Z. Li, J. Peng, Y. Mei, S. Lin, Y. Wu, O. Padon, and Z. Jia, “Quarl: A learning-based quantum circuit optimizer,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 555–582, 2024.

- [43] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 1001–1014, 2019.
- [44] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "Lightsabre: A lightweight and enhanced sabre algorithm," *arXiv preprint arXiv:2409.08368*, 2024.
- [45] M. Alam, A. Ash-Saki, and S. Ghosh, "An efficient circuit compilation flow for quantum approximate optimization algorithm," *Proceedings of the 57th ACM/EDAC/IEEE Design Automation Conference*, 2020.
- [46] Y. Jin, F. Hua, Y. Chen, A. Hayes, C. Zhang, and E. Z. Zhang, "Exploiting the regular structure of modern quantum architectures for compiling and optimizing programs with permutable operators," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ser. ASPLOS '23. New York, NY, USA: Association for Computing Machinery, 2024, p. 108–124. [Online]. Available: <https://doi.org/10.1145/3623278.3624751>
- [47] T. Forster, N. Quetschlich, and R. Wille, "Quantum circuit optimization for the fault-tolerance era: Do we have to start from scratch?" *arXiv preprint arXiv:2509.02668*, 2025.
- [48] M. E. Beverland, P. Murali, M. Troyer, K. M. Svore, T. Hoefler, V. Kliuchnikov, G. H. Low, M. Soeken, A. Sundaram, and A. Vasshillo, "Assessing requirements to scale to practical quantum advantage," *arXiv preprint arXiv:2211.07629*, 2022.
- [49] W. van Dam, M. Mykhailova, and M. Soeken, "Using azure quantum resource estimator for assessing performance of fault tolerant quantum computation," in *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 1414–1419.
- [50] D. S. Wang, A. G. Fowler, and L. C. Hollenberg, "Quantum computing with nearest neighbor interactions and error rates over 1%," *arXiv preprint arXiv:1009.3686*, 2010.
- [51] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 3, p. 032324, 2012.
- [52] N. J. Ross and P. Selinger, "Optimal ancilla-free clifford+ t approximation of z-rotations," *Quantum Inf. Comput.*, vol. 16, no. 11&12, pp. 901–953, 2016.
- [53] P. Jordan and E. Wigner, "Über das paulische äquivalenzverbot," *Zeitschrift für Physik*, vol. 47, no. 9, pp. 631–651, 1928.
- [54] S. B. Bravyi and A. Y. Kitaev, "Fermionic quantum computation," *Annals of Physics*, vol. 298, no. 1, pp. 210–226, 2002.
- [55] J. T. Seeley, M. J. Richard, and P. J. Love, "The bravyi-kitaev transformation for quantum computation of electronic structure," *The Journal of chemical physics*, vol. 137, no. 22, 2012.
- [56] L. Monbroussou, E. Z. Mamon, J. Landman, A. B. Grilo, R. Kukla, and E. Kashefi, "Trainability and expressivity of hamming-weight preserving quantum circuits for machine learning," *Quantum*, vol. 9, p. 1745, 2025.
- [57] L. Burgholzer and R. Wille, "Advanced equivalence checking for quantum circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 9, pp. 1810–1824, 2020.
- [58] E. Jang, S. Choi, and W. W. Ro, "Quixote: Improving fidelity of quantum program by independent execution of controlled gates," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [59] Y. Shi, R. Tao, X. Li, A. Javadi-Abhari, A. W. Cross, F. T. Chong, and R. Gu, "Certiq: a mostly-automated verification of a realistic quantum compiler," *arXiv preprint arXiv:1908.08963*, 2019.
- [60] J. Arora, M. Xu, S. Westrick, P. Liu, D. Li, Y. Ding, and U. A. Acar, "Local optimization of quantum circuits (extended version)," *arXiv preprint arXiv:2502.19526*, 2025.
- [61] P. Liu, J. Arora, M. Xu, and U. A. Acar, "Popqc: Parallel optimization for quantum circuits," in *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, 2025, pp. 269–283.
- [62] J. Pointing, O. Padon, Z. Jia, H. Ma, A. Hirth, J. Palsberg, and A. Aiken, "Quanto: Optimizing quantum circuits with automatic generation of circuit identities," *Quantum Science and Technology*, vol. 9, no. 4, p. 045009, 2024.
- [63] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, "Quantum circuit optimization with deep reinforcement learning," *arXiv preprint arXiv:2103.07585*, 2021.
- [64] M. Nägele and F. Marquardt, "Optimizing zx-diagrams with deep reinforcement learning," *Machine Learning: Science and Technology*, vol. 5, no. 3, p. 035077, 2024.
- [65] B. Nash, V. Gheorghiu, and M. Mosca, "Quantum circuit optimizations for nisq architectures," *Quantum Science and Technology*, vol. 5, no. 2, p. 025010, 2020.
- [66] M. Amy and M. Mosca, "T-count optimization and reed-muller codes," *IEEE Transactions on Information Theory*, vol. 65, no. 8, pp. 4771–4784, 2019.
- [67] L. E. Heyfron and E. T. Campbell, "An efficient quantum compiler that reduces t count," *Quantum Science and Technology*, vol. 4, no. 1, p. 015004, 2019.
- [68] V. Vandaele, "Lower t-count with faster algorithms," *Quantum*, vol. 9, p. 1860, 2025.
- [69] E. Younis, C. C. Iancu, W. Lavrijsen, M. Davis, and E. Smith, "Berkeley quantum synthesis toolkit (bqskit) v1.1," [Computer Software] <https://doi.org/10.11578/dc.20210603.2>, apr 2021. [Online]. Available: <https://doi.org/10.11578/dc.20210603.2>
- [70] X.-C. Wu, M. G. Davis, F. T. Chong, and C. Iancu, "Qgo: Scalable quantum circuit optimization using automated synthesis," *arXiv preprint arXiv:2012.09835*, 2020.
- [71] T. Patel, E. Younis, C. Iancu, W. de Jong, and D. Tiwari, "Quest: systematically approximating quantum circuits for higher output fidelity," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 514–528.
- [72] C. G. Kang and H. Oh, "Modular component-based quantum circuit synthesis," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 348–375, 2023.
- [73] M. Weiden, E. Younis, J. Kalloor, J. Kubiawicz, and C. Iancu, "Improving quantum circuit synthesis with machine learning," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 1–11.
- [74] A. Paradis, J. Dekoninck, B. Bichsel, and M. Vechev, "Synthetiq: Fast and versatile quantum circuit synthesis," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 55–82, 2024.
- [75] M. Weiden, J. Kalloor, J. Kubiawicz, E. Younis, and C. Iancu, "High-precision multi-qubit clifford+ t synthesis by unitary diagonalization," *arXiv preprint arXiv:2409.00433*, 2024.
- [76] A. Xu, A. Molavi, S. Tannu, and A. Albarghouthi, "Optimizing quantum circuits, fast and slow," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 777–793.
- [77] T. Hao, A. Xu, and S. Tannu, "Reducing t gates with unitary synthesis," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2026, pp. 1589–1604.
- [78] G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, "Paulihedral: A generalized block-wise compiler optimization framework for quantum simulation kernels," *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, p. 554–569, 2022. [Online]. Available: <https://doi.org/10.1145/3503222.3507715>
- [79] Y. Jin, Z. Li, F. Hua, T. Hao, H. Zhou, Y. Huang, and E. Z. Zhang, "Tetris: A compilation framework for vqa applications in quantum computing," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. Los Alamitos, CA, USA: IEEE Computer Society, jul 2024, pp. 277–292. [Online]. Available: <https://doi.ieeeecomputersociety.org/10.1109/ISCA59077.2024.00029>
- [80] Y. Liu, S. Che, J. Zhou, Y. Shi, and G. Li, "Fermihedral: On the optimal compilation for fermion-to-qubit encoding," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 382–397.
- [81] Z. Chen, J. Li, M. Guo, H. Chen, Z. Li, J. Bierman, Y. Huang, H. Zhou, Y. Liu, and E. Z. Zhang, "Genesis: A compiler for hamiltonian simulation on hybrid cv-dv quantum computers," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1583–1597.
- [82] E. Jang, H. Kim, Y. Lee, J. Kwon, Y. Huang, and W. W. Ro, "Toward scalable gate-level parallelism on trapped-ion processors with racetrack electrodes," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–17.

- [83] J. Zhou, Y. Liu, Y. Shi, A. Javadi-Abhari, and G. Li, “Bosehedral: Compiler optimization for bosonic quantum computing,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 261–276.